Goal Tracker App

3-Tier AWS Infrastructure

Complete Resource Breakdown & Deployment Workflow

Terraform | AWS us-east-2

Project Overview

This project deploys a CloudOps Goal Tracker application using a 3-tier architecture on AWS. The stack consists of a Node.js frontend, a Go backend, and a PostgreSQL database, all deployed using Terraform as Infrastructure as Code. The entire infrastructure is defined in a single main.tf file for simplicity.

Architecture Flow:
Internet -> Public ALB (port 80) -> Frontend ASG (nginx reverse proxy -> Node.js :3000)
Frontend proxies /api/* calls -> Internal ALB (port 80) -> Backend ASG (Go :8080)
Backend queries -> RDS PostgreSQL 15 (port 5432)

The frontend serves a web UI where users can add/remove goals. The backend provides REST API endpoints (GET/POST/DELETE /goals) backed by PostgreSQL. Metrics are exposed at /metrics.

Project Workflow

1. Terraform init/plan/apply provisions all 40 AWS resources
2. VPC with 8 subnets across 2 AZs isolates tiers (public, frontend, backend, database)
3. NAT Gateway gives private subnets internet access for package downloads
4. Auto Scaling Groups launch EC2 instances with user_data scripts
5. Frontend instances: install nginx, Node.js, clone repo, npm install, run server.js
6. Backend instances: install Go, clone repo, go build, run binary, connect to RDS
7. RDS PostgreSQL is provisioned with the goalsdb database
8. ALBs route traffic: Public ALB -> Frontend TG, Internal ALB -> Backend TG
9. Health checks verify instances are responsive before routing traffic
10. Bastion host provides SSH access to private instances for debugging

Resource Breakdown (main.tf)

Every resource in the configuration, explained:

Data Sources

`data.aws_ami.amazon_linux`
Fetches the latest Amazon Linux 2023 AMI ID dynamically. This ensures EC2 instances always launch with the most recent OS image, including security patches. Without this, you'd hardcode an AMI ID that becomes stale.

`data.aws_availability_zones.available`
Retrieves available AZs in the region. Used to distribute subnets across AZs (indices [0] and [1]) for high availability. Hardcoding AZ names like 'us-east-2a' would break if that AZ is unavailable.

Networking

`aws_vpc.my_vpc`
The virtual private network that isolates all resources. CIDR 10.0.0.0/16 provides 65,536 IP addresses. DNS hostname/support are enabled for internal DNS resolution (instances get private DNS names).

`aws_internet_gateway.gw`
Gateway connecting the VPC to the internet. Required for public subnets to reach the internet and for internet traffic to reach public ALB and bastion.

`aws_eip.nat`
Elastic IP (static public IP) allocated for the NAT Gateway. Must be created separately because NAT Gateway requires a pre-allocated EIP to associate.

`aws_nat_gateway.nat`
Allows instances in private subnets (frontend, backend, database) to download packages (git clone, npm install, go build) while blocking inbound internet traffic. Placed in public_subnet1 with the EIP. Depends on IGW for its route to the internet.

`aws_subnet.* (8 resources)`
8 subnets across 2 AZs: public(1,2) for ALB/bastion/NAT, frontend(1,2) for frontend ASG, backend(1,2) for backend ASG, database(1,2) for RDS. Each /24 provides 256 IPs. Two AZs ensure availability if one AZ goes down. Public subnets have map_public_ip_on_launch=true so bastion gets a public IP automatically.

`aws_route_table.public_rt / private_rt`
Public RT routes 0.0.0.0/0 -> IGW for internet access. Private RT routes 0.0.0.0/0 -> NAT Gateway so private instances can initiate outbound connections.

`aws_route_table_association.* (8 resources)`
Each subnet is explicitly associated with the correct route table. Public subnets get the public RT, all private subnets (frontend, backend, database) get the private RT.

Resource Breakdown (continued)

Security Groups

`aws_security_group.alb_sg`
Controls traffic to the public ALB. Allows HTTP(80) and HTTPS(443) from anywhere (0.0.0.0/0) since this is a public-facing app. Egress allows all traffic so ALB can forward requests to frontend instances.

`aws_security_group.frontend_sg`
Controls traffic to frontend EC2 instances. Only allows HTTP(80) from the ALB SG - no direct internet access to instances. This is the key security boundary: users can only reach frontends through the ALB.

`aws_security_group.backend_sg`
Controls traffic to backend EC2 instances. Only allows HTTP(80) from frontend SG - frontend instances can call the backend API, but nothing else can. Both frontend and backend SGs allow full egress for package downloads via NAT.

`aws_security_group.database_sg`
Controls traffic to RDS. Only allows PostgreSQL(5432) from backend SG - backend instances can query the database, but frontend and internet cannot. This implements defense-in-depth for the data tier.

`aws_security_group.bastion_sg`
Controls SSH access to the bastion host. Only allows SSH(22) from the user's IP (var.my_ip = 193.187.150.229/32). This prevents brute-force SSH attacks from the entire internet.

S3 & Random

`random_pet.suffix`
Generates a random two-word string (e.g., 'inspired-bison') to append to the S3 bucket name. S3 bucket names must be globally unique across all AWS accounts - a random suffix prevents name collisions with other accounts' buckets.

`aws_s3_bucket.probucket`
S3 bucket for future use (logs, backups, static assets). force_destroy=true allows Terraform to delete the bucket even if it contains objects, avoiding destroy failures. Originally had ALB access logs but that feature was removed due to permission issues.

Load Balancers

`aws_lb.public_alb`
Internet-facing Application Load Balancer that receives user traffic. Distributes requests across frontend instances in both public subnets/AZs. Uses alb_sg for security. Provides a single DNS endpoint regardless of how many frontend instances are running.

`aws_lb.int_alb`
Internal ALB that routes API calls from frontend to backend. Internal means it has a private IP only - not reachable from the internet. The frontend Node.js app sends /api/* requests to this ALB's DNS name, which forwards to backend instances.

`aws_lb_target_group.frontend_tg`
Routes traffic from public ALB to frontend instances on port 80 (nginx). Health check hits '/' - nginx proxies to Node.js which returns index.html. If health check fails 3 times, the instance is removed from rotation.

`aws_lb_target_group.backend_tg`
Routes traffic from internal ALB to backend instances on port 8080 (Go app). Health check hits '/health' which the Go app handles by returning 'OK'. Using a dedicated health endpoint is better than '/' because it can verify database connectivity too.

`aws_lb_listener.frontend_listener`
Listens on public ALB port 80 and forwards all traffic to frontend_tg. No host-based routing - all requests go to the same target group.

`aws_lb_listener.backend_listener`
Listens on internal ALB port 80 and forwards all traffic to backend_tg. The frontend's BACKEND_URL env var points to the internal ALB DNS, so API calls arrive here.

Resource Breakdown (continued)

RDS Database

`aws_db_subnet_group.postgres`
Groups the two database subnets into a subnet group required by RDS. RDS will place the database in one subnet and use the other for multi-AZ failover (though multi-AZ is disabled here for cost savings).

`aws_db_instance.postgres`
PostgreSQL 15 on db.t3.micro (1GB RAM, 2 vCPUs) with 20GB storage. The goalsdb database is created at provisioning. skip_final_snapshot=true means no snapshot is taken on destroy - fine for dev, dangerous for prod. publicly_accessible=false means it only accepts connections from within the VPC.

Launch Templates & ASGs

`aws_launch_template.frontend_lt`
Template for frontend instances. Uses the latest AL2023 AMI, t3.micro, frontend_sg, and the ansible-jenkins SSH key. The user_data script (from userdata_frontend.sh template) runs at boot: installs nginx/nodejs/git, clones the repo, runs npm install, creates systemd service for Node.js app on :3000, configures nginx reverse proxy :80->:3000, and starts everything. The BACKEND_URL env var is set to the internal ALB DNS at template render time.

`aws_launch_template.backend_lt`
Template for backend instances. user_data installs git/golang, clones repo, runs 'go build', creates systemd service for the Go binary on :8080, and starts it. Environment variables (DB_HOST, DB_PASSWORD, etc.) are written to an env file at template render time from RDS attributes.

`aws_autoscaling_group.frontend_asg`
Manages frontend instance count: min=1, max=2, desired=1. Uses ELB health checks - if the frontend_tg health check fails, the instance is terminated and replaced. force_delete=true allows Terraform to destroy the ASG even with running instances. Spans both frontend subnets for AZ redundancy.

`aws_autoscaling_group.backend_asg`
Same pattern for backend: min=1, max=2, desired=1. Attached to backend_tg for health checks. Spans both backend subnets.

Bastion

`aws_instance.bastion`
A t2.micro EC2 in the public subnet with a public IP. Only accessible via SSH from the user's IP. Used to SSH into private frontend/backend instances for debugging. The ansible-jenkins key pair must exist in the AWS account or instance creation fails.

Issues Faced & Solutions

1. ALB Access Logs S3 Permission Error

Problem: The access_logs block on the public ALB pointed to an S3 bucket. AWS requires a specific bucket policy granting the ELB service principal (delivery.logs.amazonaws.com) write access. Despite correctly configuring the policy with the proper principal, GetBucketAcl action, and account-scoped ARN, the ALB kept returning 'Access Denied for bucket'.

Solution: Removed the entire access_logs block from the ALB, along with the S3 bucket policy, ownership controls, and public access block. The S3 bucket is retained without logging. This was the cleanest path forward after extensive debugging of the service principal.

2. Default Nginx Page Instead of App

Problem: After first successful apply, the ALB returned the default nginx welcome page. The user_data script only ran 'dnf install nginx' and started it - no application code was ever deployed. The launch templates had no reference to the goal tracker repo.

Solution: Rewrote both user_data scripts to: (1) clone the GitHub repo with the full app, (2) install dependencies and build binaries, (3) create systemd services, (4) configure nginx as reverse proxy. The frontend now serves the goal tracker UI, and the backend runs the Go API.

3. Nested Heredoc Indentation in user_data

Problem: The Terraform template used <<-EOF with 4-space indentation. Inside, shell heredocs (for systemd service files, nginx config) used unquoted delimiters with the same leading spaces. The shell requires heredoc closing delimiters at column 0 - the spaces prevented matching, causing the entire script to hang/fail at cloud-init.

Solution: Extracted user_data into separate template files (userdata_frontend.sh, userdata_backend.sh) using Terraform's templatefile() function. Standalone files have no indentation conflicts. Environment variables are written with echo (line by line) instead of heredocs to avoid the issue entirely.

4. PostgreSQL vs MySQL Port Mismatch

Problem: The database security group allowed MySQL (port 3306) but the Go backend connects to PostgreSQL (port 5432). There was no RDS instance defined at all in the original config.

Solution: Added aws_db_subnet_group and aws_db_instance for PostgreSQL 15. Changed database_sg to port 5432. Updated backend target group health check to /health on port 8080. Added RDS endpoint to outputs.

Deployment Outputs

```
vpc_id = vpc-020a35e4490743c83
public_alb_dns = public-alb-1272582037.us-east-2.elb.amazonaws.com
internal_alb_dns = internal-int-alb-270528909.us-east-2.elb.amazonaws.com
bastion_public_ip = 52.15.83.107
bastion_instance_id = i-04fa14d4b268b6613
nat_gateway_public_ip = 16.59.108.218
rds_endpoint = goal-tracker-db.c9aya44ew7tw.us-east-2.rds.amazonaws.com
frontend_target_group_arn = arn:aws:elasticloadbalancing:.../frontend-tg/...
backend_target_group_arn = arn:aws:elasticloadbalancing:.../backend-tg/...
```

Instance Health Status

```
frontend-asg: i-0cb8f1d73ad4de29e  InService  Healthy
backend-asg:  i-0122a70de6840a710  InService  Healthy
```

Result: App is Live

URL: http://public-alb-1272582037.us-east-2.elb.amazonaws.com

Status: HTTP 200 OK

Application: CloudOps Goal Tracker

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>CloudOps Goal Tracker</title>
    <link href="https://fonts.googleapis.com/css2?family=Poppins:wght@300;400;500;600&display=swap" rel="sty
    <style>
        :root {
            --primary: #4361ee;
            --primary-dark: #3a56d4;
            --primary-light: #eef0ff;
            --secondary: #ff6b6b;
            --secondary-dark: #ff5252;
            --secondary-light: #ffeded;
            --text: #333;
            --text-light: #666;
            --bg: #f0f2f5;
            --card-bg: #ffffff;
            --border: #e0e0e0;
            --success: #4caf50;
        }

        /* Dark theme variables */
        [data-theme="dark"] {
            --primary: #6366f1;
            --primary-dark: #4f46e5;
            --primary-light: #1e1b4b;
            --secondary: #ef4444;
            --secondary-dark: #dc2626;
            --secondary-light: #450a0a;
            --text: #f8fafc;
            --text-light: #cbd5e1;
            --bg: #0f172a;
            --card-bg: #1e293b;
            --border: #475569;
            --success: #10b981;
        }

        * {
            margin: 0;
            padding: 0;
```

```
    box-sizing: border-box;
}

/* Better background gradient for the body */
body {
    font-family: 'Poppins', sans-serif;
    background: linear-gradient(135deg, #8BC6EC 0%, #9599E2 100%);
    background-attachment: fixed;
    color: var(--text);
    line-height: 1.6;
    min-height: 100vh;
    transition: all 0.5s ease;
}

/* Dark theme background */
[data-theme="dark"] body {
    background: linear-gradient(135deg, #000000 0%, #1a1a1a 50%, #2d2d2d 100%);
}       /* Improved container glassmorphism - REMOVED float animation */
.container {
    width: 90%;
    max-width: 800px;
    margin: 40px auto;
    padding: 30px;
    background: rgba(255, 255, 255, 0.15);
    backdrop-filter: blur(20px);
    -webkit-backdrop-filter: blur(20px);
    border-radius: 24px;
    border: 1px solid rgba(255, 255, 255, 0.3);
    box-shadow:
        0 10px 30px rgba(0, 0, 0, 0.1),
        0 1px 2px rgba(255, 255, 255, 0.3) inset;
    transition: all 0.5s cubic-bezier(0.175, 0.885, 0.32, 1.275);
    will-change: transform, box-shadow;
    overflow: hidden;
    position: relative;
}

/* Dark theme container adjustments - keep original glassmorphism */
[data-theme="dark"] .container {
    background: rgba(255, 255, 255, 0.15);
    border: 1px solid rgba(255, 255, 255, 0.3);
    box-shadow:
        0 10px 30px rgba(0, 0, 0, 0.1),
        0 1px 2px rgba(255, 255, 255, 0.3) inset;
}

/* Add subtle background pattern */
.container::before {
    content: '';
    position: absolute;
    top: 0;
    left: 0;
    right: 0;
    bottom: 0;
    background: linear-gradient(45deg, transparent 49%, rgba(255, 255, 255, 0.05) 50%, transparent 5
    background-size: 20px 20px;
    pointer-events: none;
    z-index: -1;
}

/* Improved header styling */
.header {
    display: flex;
    align-items: center;
    justify-content: center;
    margin-bottom: 40px;
    position: relative;
    padding-bottom: 20px;
    border-bottom: 1px solid rgba(255, 255, 255, 0.3);
}

/* Enhanced logo styling with better glow effect */
.logo {
    width: 50px;
    height: 50px;
    margin-right: 15px;
    border-radius: 15px;
    background: linear-gradient(45deg, var(--primary), var(--primary-dark));
    box-shadow:
        0 4px 15px rgba(67, 97, 238, 0.3),
        0 0 15px rgba(67, 97, 238, 0.5);
    padding: 8px;
    display: flex;
    align-items: center;
    justify-content: center;
    animation: pulse-glow 2s infinite;
    will-change: transform, box-shadow;
    position: relative;
    overflow: hidden;
```

Showing 131 lines of HTML (total 27565 bytes)